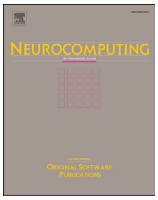
Neurocomputing 678 (2026) 133192



Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom





Adaptive agentic meta-controller (AAMC): A deep reinforcement learning framework for intelligent SLM/LLM orchestration

Gaith Rjoub[a, b], Jamal Bentahar[c, b,*], Shahed Almobydeen[d, a], Witold Pedrycz[e, f, g], Ahmad Irjoob[h]

[a] Faculty of Information Technology, Aqaba University of Technology, Aqaba, Jordan
[b] Concordia Institute for Information Systems Engineering, Concordia University, Montreal, Canada
[c] Department of Computer Science, 6G Research Center, Khalifa University, Abu Dhabi, United Arab Emirates
[d] Management Information System Department, Al-Hussein Bin Talal University, Ma'an, Jordan
[e] University of Alberta, 116 St and 85 Ave, Edmonton, Canada
[f] Polish Academy of Sciences, Warsaw, 00-901, Poland
[g] Istinye University, Vadistanbul 4A Blok, Sariyer/Istanbul, 34396, Turkey
[h] Faculty of Artificial Intelligence, Al-Balqa Applied University, Assalt, Jordan

ARTICLE INFO

Communicated by Z. Wang

Keywords:
Large language models
Small language models
Reinforcement learning
Agentic systems
Model orchestration
Multi-objective optimization
Cost-effective AI

ABSTRACT

The rapid proliferation of agentic AI systems has been dominated by Large Language Models (LLMs), but their substantial operational costs and high latency present significant barriers to widespread adoption. In response, the research community has increasingly turned to Small Language Models (SLMs), which offer a compelling combination of efficiency, task-specificity, and cost-effectiveness. This paper introduces the Adaptive Agentic Meta-Controller (AAMC), a deep reinforcement learning (RL) framework designed for intelligent SLM/LLM orchestration. The AAMC transforms the model selection problem into a principled, multi-objective optimization task, learning a dynamic policy that routes queries to the most appropriate model—preferring SLMs for routine tasks and escalating to LLMs only when necessary. Our framework features a Task Complexity Estimator (TCE) and an RL-based Router (RLR) that collaboratively balance the trade-offs between performance, cost, and latency. We conduct extensive experiments in a high-fidelity simulation environment, demonstrating that the AAMC achieves a task success rate comparable to an LLM-only approach while reducing operational costs by over 70% and significantly improving inference latency. We further introduce a comprehensive set of experiments on robustness, scalability, and fairness, including new ablation studies on the impact of the TCE and the sensitivity to user preferences, alongside a detailed complexity analysis and a discussion of real-world deployment. We further introduce a comprehensive set of experiments on robustness, scalability, and fairness, alongside a detailed complexity analysis and a real-world enterprise deployment case study. We also release code and experiments to support reproducibility.

1. Introduction

The rise of agentic artificial intelligence[1] marks a paradigm shift in how we interact with complex digital systems [10,14,18,29]. These autonomous agents, powered by language models, are increasingly capable of performing complex tasks, from software development to scientific discovery [8,25]. However, the vast majority of these systems have been built on the foundation of Large Language Models (LLMs), which, despite their impressive capabilities, come with significant drawbacks. The operational costs of LLMs are substantial, and their high inference latency makes them unsuitable for many real-time applications [9,36–38].

* Corresponding author at: Department of Computer Science, 6G Research Center, Khalifa University, Abu Dhabi, United Arab Emirates.
Email addresses: grjoub@aut.edu.jo (G. Rjoub), jamal.bentahar@ku.ac.ae (J. Bentahar), salmobydeen@aut.edu.jo (S. Almobydeen), wpedrycz@ualberta.ca (W. Pedrycz), ahmad.irjoob@bau.edu.jo (A. Irjoob).

[1] We define "agentic AI" as autonomous systems capable of perception, reasoning, and acting upon their environment, often leveraging tools to achieve complex goals.

https://doi.org/10.1016/j.neucom.2026.133192
Received 11 October 2025; Received in revised form 11 February 2026; Accepted 26 February 2026
Available online 28 February 2026
0925-2312/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

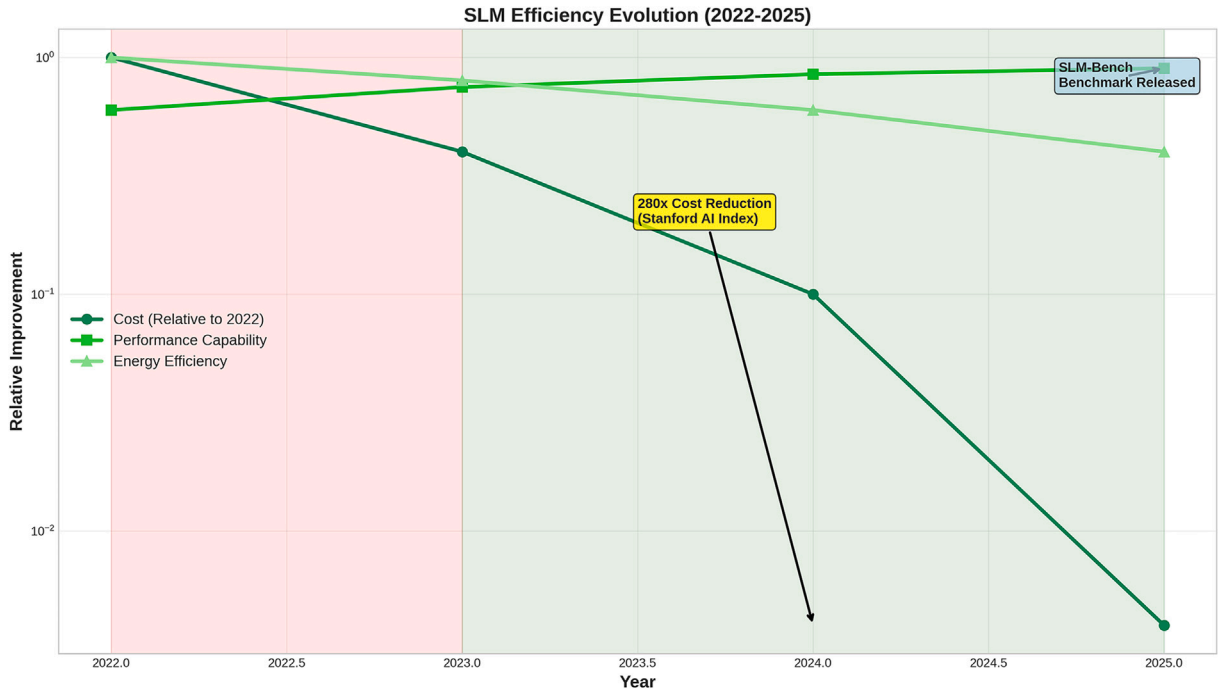


Fig. 1. A timeline illustrating the rapid evolution of SLM efficiency from 2022 to 2025, based on data from the Stanford AI Index and other research.

In response to these challenges, the research community has witnessed a surge of interest in Small Language Models (SLMs) in 2024 and 2025 [3,28,34,40]. These models offer a compelling alternative, providing greater efficiency in terms of computational resources and energy consumption, and their lower latency makes them ideal for a wide range of applications. As the 2025 AI Index from Stanford HAI [15] highlights, the inference cost to reach GPT-3.5–level performance fell by more than 280× (from about $20 to $0.07 per million tokens) between Nov 2022 and Oct 2024, a trend driven by increasingly capable small models. This dramatic evolution in efficiency is illustrated in Fig. 1. This trend, coupled with the fact that open-weight models are rapidly closing the performance gap with their closed-source counterparts, is democratizing access to advanced AI capabilities. Beyond cost and latency, the shift towards efficient model orchestration is also motivated by the increasing need for Green AI [30], as reducing reliance on energy-intensive LLMs can significantly lower the carbon footprint of large-scale AI systems.

However, the transition to an SLM-centric paradigm is not without its challenges. While SLMs excel at specific tasks, they often lack the broad, general-purpose reasoning capabilities of LLMs [12,20,33]. This has led to the emergence of a new research frontier: the intelligent orchestration of heterogeneous language models [11,17,23,39]. The goal is to create systems that can dynamically route queries to the most appropriate model, leveraging the efficiency of SLMs for routine tasks while reserving the power of LLMs for more complex challenges. This fundamental trade-off is visualized in Fig. 2. The comparison illustrates the trade-offs between SLMs and LLMs across six key metrics. SLMs demonstrate clear advantages in latency, cost efficiency, energy efficiency, and deployment ease due to their smaller size and reduced computational demands, making them well-suited for real-time and resource-constrained environments. They also perform strongly in task-specific specialization when fine-tuned. In contrast, LLMs achieve superior performance in terms of generalization and reasoning capability [6,13] but at the expense of higher latency, greater energy consumption, and increased deployment costs. This highlights the complementary nature of the two model classes, where SLMs excel in efficiency and practicality, while LLMs lead in raw performance and versatility.

This paper introduces the Adaptive Agentic Meta-Controller (AAMC), a novel framework that addresses this challenge by framing the model selection problem as a multi-objective deep reinforcement learning (RL) task. The AAMC learns a dynamic routing policy that optimizes a combination of performance, cost, and latency, thereby enabling the creation of agentic AI systems that are both powerful and economically viable. Our work is among the first to apply multi-objective RL to the problem of inter-model routing in agentic systems, providing a significant step towards the future of AI-driven systems.

1.1. Contributions

This work makes several novel contributions to the field of agentic AI orchestration:

1. We introduce the Adaptive Agentic Meta-Controller (AAMC), a novel framework that intelligently orchestrates between Small and Large Language Models (SLMs and LLMs).
2. We formalize the dynamic model selection problem as a multi-objective RL task, learning a policy that explicitly balances the trade-offs between performance, operational cost, and inference latency.
3. We propose a two-component architecture featuring a Task Complexity Estimator (TCE) and an RL-based Router (RLR) that work in concert to make informed, dynamic routing decisions.
4. We provide extensive empirical validation in a simulated enterprise environment, demonstrating that the AAMC can reduce operational costs by over 70% while maintaining a task success rate comparable to a costly LLM-only approach. We also include new ablation and sensitivity analyses to further validate our design choices.

1.2. Paper organization

The remainder of this paper is structured as follows. Section 2 reviews the related work on language models, dynamic routing, and the application of reinforcement learning for resource management.

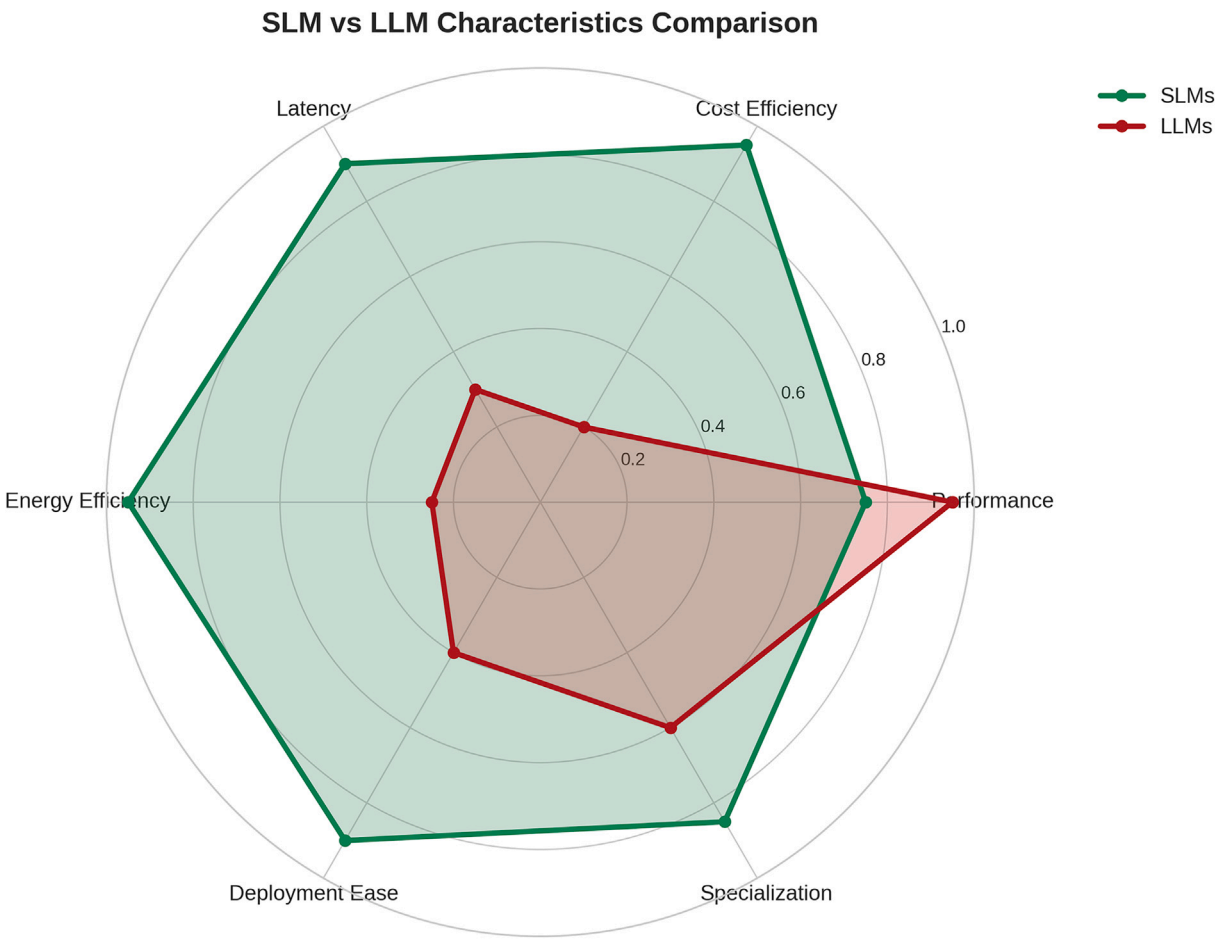


Fig. 2. A radar chart comparing the key characteristics of SLMs and LLMs, highlighting the trade-offs between performance, cost, and efficiency.

Section 3 provides a deep, technical description of our proposed AAMC framework, including the Task Complexity Estimator and the RL-based Router. Section 4 details our comprehensive experimental setup, and presents the full results of our evaluation, including baseline comparisons and analyses of robustness, scalability, and fairness. Finally, Section 5 discusses the limitations, deployment considerations, and future directions of our work, and Section 6 concludes the paper.

2. Related work

The research landscape in agentic AI has been characterized by a rapid evolution, with a notable shift in focus from monolithic LLMs to more heterogeneous and efficient systems. This section reviews the key trends and recent literature that provide the context for our work on the AAMC framework.

2.1. The rise of small language models (SLMs)

The years 2024 and 2025 have been marked by a surge of interest in SLMs. As highlighted in the 2025 AI Index Report from Stanford HAI, the increasing capability of small models has been a primary driver of the dramatic reduction in AI inference costs. This trend is further substantiated by the work of Belcak et al. [3], who argue that SLMs are not only sufficiently powerful for many agentic tasks but are also inherently more suitable and economically viable than their larger counterparts. The SLM-Bench framework, introduced by Pham et al. [24], provides the first comprehensive benchmark for evaluating SLMs across multiple dimensions, including accuracy, efficiency, and environmental impact. Their findings confirm the existence of a diverse set of trade-offs, with some SLMs excelling in accuracy and others in energy efficiency. This body of work underscores the growing consensus that the future of agentic AI lies in the intelligent orchestration of a diverse pool of language models, with SLMs playing a central role.

2.2. Dynamic model routing and mixture-of-experts

The challenge of efficiently orchestrating multiple language models has led to two primary strategies: routing and Hierarchical Inference (HI), as systematically characterized in a recent survey by Behera et al. [2]. Our AAMC framework is a novel form of routing, where the goal is to select the most suitable model from a pool for a given query. This inter-model routing differs from intra-model approaches like Mixture-of-Experts (MoE), which activate different components within a single, large model.

Recent work has produced several powerful inter-model routing frameworks. A state-of-the-art example is RouterDC [5], which frames the routing problem as a representation learning task. It uses a novel dual contrastive learning method to learn a router that maps a query to the most suitable LLM. Another advanced system is ECO-LLM [22], which treats the problem as a joint optimization of the entire serving pipeline, selecting not just a model but also the best context retrieval and query preprocessing components for a task. However, these approaches primarily rely on supervised learning paradigms that map static query features to a model, optimizing implicitly for accuracy or throughput. In contrast, the AAMC framework introduces a fundamental shift by treating model selection as a sequential decision-making problem under uncertainty. As summarized in Table 1, our approach differs in

Table 1
Comparison of AAMC with state-of-the-art routing frameworks.

Feature	AAMC (Ours)	RouterDC [5]	ECO-LLM [22]
Learning Paradigm	Reinforcement Learning	Supervised Learning	Supervised Learning
Primary Objective	Multi-Objective	Accuracy	Accuracy
Dynamic State Awareness	Yes (Queues, Load)	No	No
Cost/Latency Optimization	Explicit	Implicit	Implicit
Adaptability	High (Online Learning)	Low (Offline Trained)	Low (Offline Trained)

three critical ways. First, unlike supervised routers that are trained offline, AAMC utilizes RL to adapt its policy online. Second, while prior works treat system state as transparent or static, AAMC is explicitly state-aware, factoring in dynamic variables such as current queue lengths and model loads. Third, rather than optimizing for a single metric like accuracy, AAMC employs a multi-objective reward function that explicitly balances performance against operational cost and latency.

2.3. Reinforcement learning for resource management

The application of advanced machine learning to resource management is a well-established field, with recent work increasingly focused on the unique challenges of orchestrating language models. Methodologies range from supervised approaches, such as using a fine-tuned lightweight LLM as an intelligent scheduler [21,32], to dynamic, policy-based systems. A prominent example is the use of metareasoning, which employs a high-level meta-policy to make real-time decisions, such as dynamically switching between a cloud-based LLM and an edge-based LLM based on network conditions [1,19].

RL offers a particularly powerful paradigm for these problems, as it can learn optimal policies through direct interaction with an environment. The synergy between LLMs and RL is an active and diverse research area. In some cases, LLMs are used to improve the RL process itself. For example, the LEHR framework leverages an LLM's reasoning capabilities to address the critical challenge of reward design in multi-agent RL (MARL) by generating and evolving complex, multi-layered reward functions [35]. In other work, RL is used as a powerful fine-tuning mechanism to optimize an LLM for a specific generative task, such as using Proximal Policy Optimization (PPO) to generate toxic text paraphrases for data augmentation [4].

Our AAMC framework contributes to this literature by applying multi-objective RL directly to the resource management problem of model selection. While other approaches use LLMs to improve RL algorithms or use RL to fine-tune LLM behavior, the AAMC uses RL as the core decision-making engine to orchestrate a heterogeneous system of SLMs and LLMs. To our knowledge, the AAMC is the first framework to apply a preference-conditioned, multi-objective RL policy to the specific challenge of intelligent SLM/LLM orchestration.

3. The AAMC framework

The AAMC framework is designed to address the challenge of intelligent SLM/LLM orchestration by framing the model selection problem as a multi-objective deep RL task. The framework consists of two key components: a Task Complexity Estimator (TCE) and a deep Reinforcement Learning-based Router (RLR) as shown in Fig. 3. These components work in concert to dynamically route incoming queries to the most appropriate model in a heterogeneous pool of SLMs and LLMs, optimizing for a combination of performance, cost, and latency.

3.1. Task complexity estimator (TCE)

The TCE is a crucial prerequisite for the router, designed to provide a rich, real-time assessment of each incoming query's characteristics. It functions as a feature extraction frontend, transforming a raw text query into a structured vector that the RLR can use to make an informed decision. To achieve this, we implement the TCE as a fine-tuned encoder model (e.g., a distilled BERT variant) with multiple prediction heads, allowing it to perform several sub-tasks simultaneously.

3.1.1. Mathematical formulation

Let an incoming query be a sequence of tokens $Q = \{q_1, q_2, \ldots, q_n\}$. The TCE, denoted by an encoder function E_ψ with parameters ψ, processes this sequence to produce a final pooled representation, h_Q, typically the hidden state of the '[CLS]' token.

$$h_Q = E_\psi(Q) \tag{1}$$

This representation is then fed into three distinct, parallel prediction heads to produce the final feature vector z_Q:

1. **Complexity Score (c_Q):** A regression head that outputs a continuous value representing the task's difficulty. This is modeled as a linear layer with a sigmoid activation to bound the score between 0 and 1.

$$c_Q = \sigma(W_c h_Q + b_c) \tag{2}$$

2. **Task Category Embedding (e_Q):** A classification head that maps the query to one of k predefined task categories (e.g., 'Simple Q&A', 'Code Generation'). The embedding e_Q is the pre-activation logit vector.

$$e_Q = W_{cat} h_Q + b_{cat} \quad \text{where } e_Q \in \mathbb{R}^k \tag{3}$$

3. **Tool-Use Indicator (t_Q):** A binary classification head of predicts the probability that whether the task requires an external tool.

$$t_Q = \sigma(W_{tool} h_Q + b_{tool}) \tag{4}$$

The final feature vector, z_Q, is the concatenation of these three outputs, $z_Q = \text{concat}(c_Q, e_Q, t_Q)$, which serves as a key part of the state representation for the RLR.

3.1.2. Training procedure

The TCE is trained on a labeled dataset D_{TCE} of queries, where each query is annotated with a ground-truth complexity score, task category, and tool-use flag. To train the multiple heads simultaneously, we use a multi-task loss function, $\mathcal{L}_{TCE}$, which is a weighted sum of the losses from each head:

$$\mathcal{L}_{TCE} = \lambda_c \mathcal{L}_c + \lambda_{cat} \mathcal{L}_{cat} + \lambda_t \mathcal{L}_t \tag{5}$$

where $\mathcal{L}_c$ is the Mean Squared Error loss for complexity regression, $\mathcal{L}_{cat}$ is the Cross-Entropy loss for category classification, and $\mathcal{L}_t$ is the Binary Cross-Entropy loss for the tool-use prediction. The λ terms are hyperparameters that balance the contribution of each task. The full training procedure is detailed in Algorithm 1. To establish the ground-truth labels for this training process (c^*, cat^*, t^*), a synthetic annotation pipeline was employed. The complexity score c_i^* was derived empirically by assessing the success rates of the model pool on the generated queries; queries solvable only by the largest LLMs were assigned high complexity scores. Crucially, the task category cat_i^* and tool-use flag t_i^* were automatically annotated by a Teacher LLM (GPT-4) during generation, ensuring strict consistency across the dataset.

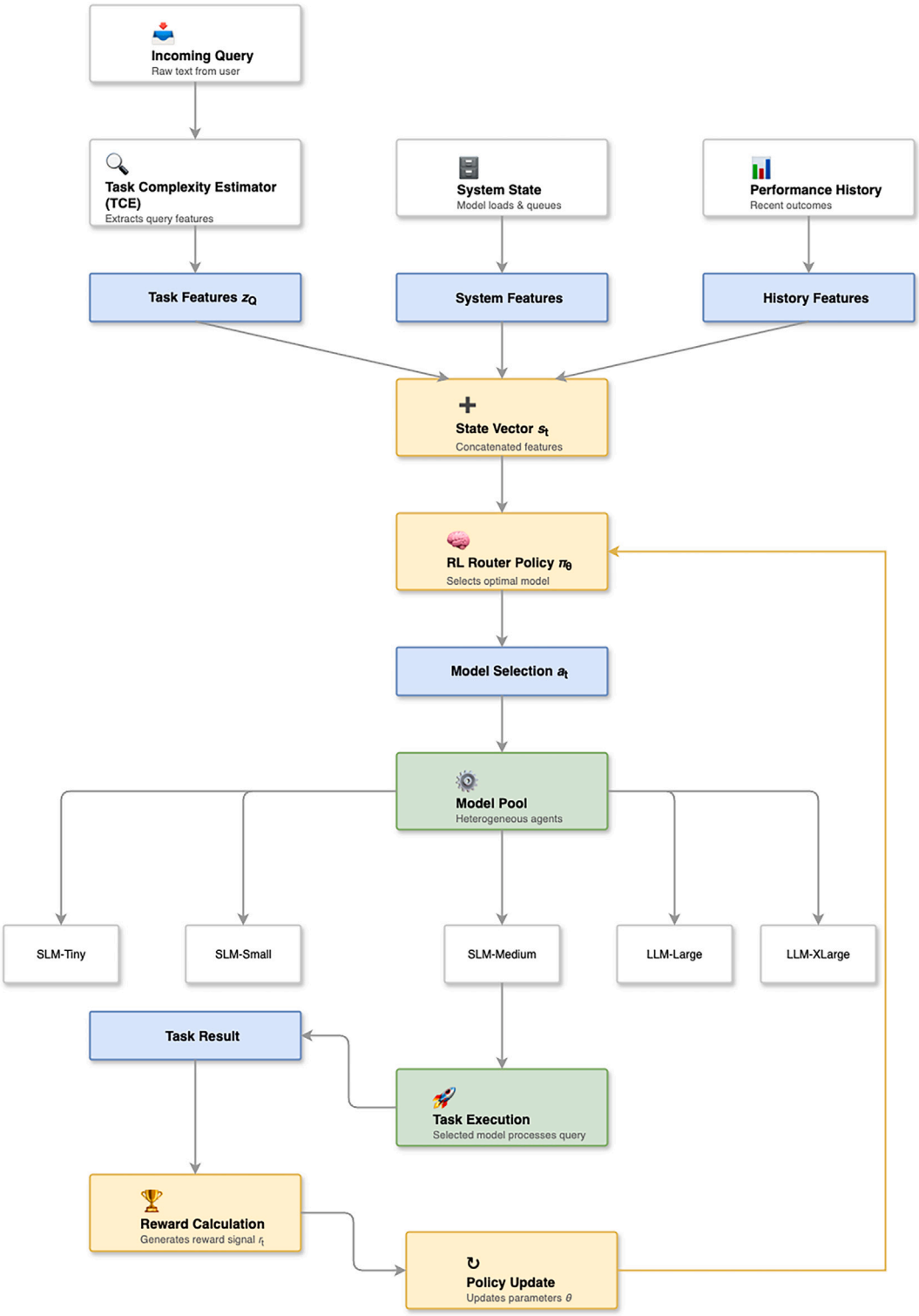


Fig. 3. The AAMC framework architecture, illustrating the flow from task input to model selection.

3.2. Deep reinforcement learning-based router (RLR)

The RLR is the core decision-making engine of the AAMC framework. It is a deep RL agent that learns a dynamic routing policy, π_θ, which maps the current state (including the query features from the TCE) to a probability distribution over the available models. The model with the highest probability is then selected to handle the task. The RLR's goal is to learn a policy that maximizes a cumulative multi-objective reward signal, effectively balancing performance, cost, and latency.

3.2.1. Multi-objective MDP formulation

To properly handle the competing objectives, we formalize the SLM/LLM orchestration problem as a multi-objective MDP $\mathcal{M} = (S, \mathcal{A}, \vec{\mathcal{R}}, P, \gamma, \rho_0, \Omega)$.

Algorithm 1 TCE Multi-Task Training Procedure.

Input: Labeled dataset $D_{TCE} = \{(Q_i, c_i^*, cat_i^*, t_i^*)\}_{i=1}^{N}$
Input: Loss weights $\lambda_c, \lambda_{cat}, \lambda_t$
Initialize: TCE model E_ψ with pre-trained weights; prediction heads W_c, W_{cat}, W_{tool}
for each epoch **do**
  for each batch $(Q_b, c_b^*, cat_b^*, t_b^*) \in D_{TCE}$ **do**
    $h_{Q_b} \leftarrow E_\psi(Q_b)$ ▷ Get pooled representation
    $\hat{c}_b \leftarrow \sigma(W_c h_{Q_b} + b_c)$ ▷ Predict complexity
    $\hat{e}_b \leftarrow W_{cat} h_{Q_b} + b_{cat}$ ▷ Predict category logits
    $\hat{t}_b \leftarrow \sigma(W_{tool} h_{Q_b} + b_{tool})$ ▷ Predict tool-use
    $\mathcal{L}_c \leftarrow \text{MSELoss}(\hat{c}_b, c_b^*)$
    $\mathcal{L}_{cat} \leftarrow \text{CrossEntropyLoss}(\hat{e}_b, cat_b^*)$
    $\mathcal{L}_t \leftarrow \text{BCELoss}(\hat{t}_b, t_b^*)$
    $\mathcal{L}_{TCE} \leftarrow \lambda_c \mathcal{L}_c + \lambda_{cat} \mathcal{L}_{cat} + \lambda_t \mathcal{L}_t$ ▷ Compute total loss
    Perform backpropagation on $\mathcal{L}_{TCE}$ to update ψ, W_c, W_{cat}, W_{tool}
  end for
end for
return Trained TCE model

- **State Space ($\mathcal{S}$):** The state $s_t \in \mathcal{S}$ provides a comprehensive snapshot of the system at timestep t. It includes the task features from the TCE (z_Q) and richer system-state features like the queue length (q_i) and time-averaged load ($\bar{l}_i$) for each of the M models in the pool to provide a stable, informative signal:

$$s_t = \text{concat}(z_Q, q_1, \ldots, q_M, \bar{l}_1, \ldots, \bar{l}_M) \tag{6}$$

- **Action Space ($\mathcal{A}$):** The action space is the discrete set of all available models in the pool, $\mathcal{A} = \{m_1, m_2, \ldots, m_M\}$, where each m_i can be an SLM or an LLM. The action a_t is the selection of one model from this set.
- **Vector Reward Function ($\vec{\mathcal{R}}$):** We use a vector-valued reward function to represent the multiple, often competing, objectives. For an action $a_t = m_i$, the reward vector $\vec{r}_t \in \mathbb{R}^3$ is:

$$\vec{r}_t = \vec{r}(s_t, a_t) = \begin{bmatrix} \text{Success}(m_i, Q) \\ -\text{Cost}(m_i) \\ -\text{Latency}(m_i) \end{bmatrix} \tag{7}$$

  where components are normalized to comparable scales and each component of the vector corresponds to one of the system's core objectives. The reward vector is constructed such that maximizing a linear combination of its components achieves the desired system behavior (i.e., maximizing success while minimizing cost and latency).
- **Transition Dynamics (P):** The transition to the next state s_{t+1} is a function of both the stochastic arrival of a new query Q_{t+1} and the updated system state (e.g., model queue lengths) resulting from action a_t being taken at time t.
- **Discount γ and initial-state distribution ρ_0.**
- **Preference Space (Ω):** To navigate the trade-offs, we define a preference space Ω, where a preference vector $\omega \in \Omega$ ($\|\omega\|_1 = 1$) specifies the desired trade-off (e.g., $\omega = [0.7, 0.2, 0.1]$ for a performance-focused policy). For a given $\omega \in \Omega$, the objective is to find:

$$\pi_\omega^* \in \arg\max_\pi J_\omega(\pi) \quad \text{where} \quad J_\omega(\pi) = \mathbb{E}_{\substack{s_0 \sim \rho_0, \\ a_t \sim \pi(\cdot|s_t,\omega), \\ s_{t+1} \sim P(\cdot|s_t,a_t)}} \left[\sum_{t=0}^{\infty} \gamma^t \, \omega \cdot \vec{r}_t \right].$$

3.2.2. Policy optimization and inference

The RLR is implemented using an actor-critic architecture, which consists of two neural networks: a policy network (the actor) $\pi_\theta(a_t|s_t, \omega)$ with parameters θ, and a value network (the critic) $V_\phi(s, \omega) \approx \mathbb{E}[\sum_{t \geq 0} \gamma^t \omega \cdot \vec{r}_t \mid s, \omega]$ with parameters ϕ. The actor's role is to select an action (a model), while the critic's role is to estimate the expected return from a given state, which helps to guide the actor's learning.

Algorithm 2 RLR training procedure (preference-conditioned PPO).

Initialize: Policy network parameters θ, value network parameters ϕ.
for iteration = 1, 2, … **do**
  Initialize empty set of trajectories $\mathcal{D} \leftarrow \emptyset$
  for actor = 1, 2, …, N **do**
    Sample a preference vector $\omega \sim$ Uniform
    Collect trajectory τ by running policy $\pi_\theta(a|s, \omega)$ in the environment.
    For each step $(s_t, a_t, \vec{r}_t)$ in τ, compute scalar reward $r_t = \omega \cdot \vec{r}_t$.
    Add τ to $\mathcal{D}$.
  end for
  Compute advantage estimates $\hat{A}_t$ for all steps in $\mathcal{D}$ using GAE.
  for epoch = 1, 2, …, K **do**
    Optimize surrogate objective $\mathcal{L}(\theta, \phi)$ with respect to θ, ϕ using minibatch gradient ascent.
  end for
end for

Training procedure. We train these networks using the PPO algorithm, chosen for its stability and state-of-the-art performance. The training process, formalized in Algorithm 2, is an iterative loop. In each iteration, we collect a batch of experience trajectories by running the current policy in the simulation environment. For each step in these trajectories, the vector reward $\vec{r}_t$ is scalarized using a sampled preference vector, $r_t = \omega \cdot \vec{r}_t$.

With this data, we then compute the advantage estimates, $\hat{A}_t$, using Generalized Advantage Estimation (GAE), which provides a stable, low-variance estimate. Finally, we update the network parameters by optimizing a composite loss function via gradient ascent. The full loss function includes the PPO clipped surrogate objective for the policy, a loss term for the value function, and an entropy bonus to encourage exploration:

$$\mathcal{L}(\theta, \phi) = \hat{\mathbb{E}}_t[\mathcal{L}^{CLIP}(\theta) - c_1 \mathcal{L}^{VF}(\phi) + c_2 S[\pi_\theta](s_t, \omega)] \tag{8}$$

where $\mathcal{L}^{CLIP}$ is the core PPO objective, $\mathcal{L}^{VF}$ is the squared-error loss for the value network, S is an entropy bonus, and c_1, c_2 are coefficients.

Inference procedure. During inference, the fully trained AAMC framework processes an incoming query to select the optimal model according to a given operational preference. This procedure, formalized in Algorithm 3, is highly efficient. The TCE first extracts features from the query, which are combined with the current system state. The RLR then uses its learned policy to select the most appropriate model.

Algorithm 3 AAMC Inference Procedure for a Single Query.

Input: New query Q, current system state, operational preference vector ω
Output: Selected model m_i
$z_Q \leftarrow \text{TCE}(Q)$ ▷ Get features from the trained TCE
$s_t \leftarrow \text{concat}(z_Q, \text{SystemState})$ ▷ Construct the full state vector
ActionProbabilities $\leftarrow \pi_\theta(a|s_t, \omega)$ ▷ Get action probabilities from the RLR
$a_t \leftarrow \text{argmax}(\text{ActionProbabilities})$ ▷ Select the model with the highest probability
return a_t

4. Experimental evaluation

4.1. Experimental setup

To empirically validate the AAMC framework, we designed a comprehensive set of experiments within a high-fidelity simulation environment. The objective of our evaluation is to rigorously assess the AAMC's ability to orchestrate a heterogeneous pool of models, balancing the competing objectives of task success, operational cost, and inference latency against several strong baseline strategies. Our simulation is designed to model a realistic enterprise environment with a continuous stream of diverse tasks.

4.1.1. Simulation environment and real-world fidelity

To ensure a rigorous evaluation and high-fidelity results, the simulation environment was executed on a workstation equipped with an NVIDIA RTX 3090 GPU, an Intel Core i9-12900K CPU, and 64GB of RAM. All framework components, including the TCE and RLR, were implemented using PyTorch 2.1 with CUDA 12.1. The simulation captures the critical stochastic dynamics of a multi-model serving system, including probabilistic task arrivals and model-specific processing queues.

While a simulation cannot perfectly replicate a live deployment, we ensured fidelity through three key mechanisms:

- **Traffic Modeling:** User request patterns are modeled using a Poisson distribution, a standard method for simulating asynchronous arrival rates in web services.
- **Congestion Dynamics:** We explicitly model inference queues, allowing the system to simulate latency spikes during high-load periods, a critical factor for evaluating dynamic routing.
- **Economic Realism:** Cost and baseline latency figures are derived from 2024 commercial API pricing (e.g., OpenAI, Azure) and standard benchmarks (e.g., HuggingFace Open LLM Leaderboard) to ensure valid economic trade-offs.

We believe this setup provides a credible proxy for real-world performance, enabling controlled experimentation with reproducible conditions.

4.1.2. Dataset and model pool

Our experiments use a large-scale dataset of $100,000$ queries, synthetically generated to cover a wide range of topics and complexity levels, and annotated as described in Section 3.1.2. The dataset statistics are provided in Table 2. The environment contains a heterogeneous pool of five models with varying capabilities and costs, mirroring the landscape of modern language models: three SLMs (`SLM-Tiny`, `SLM-Small`, `SLM-Medium`) and two LLMs (`LLM-Large`, `LLM-XLarge`).

4.1.3. Training hyperparameters

Key hyperparameters for both the TCE and RLR components were selected through a combination of literature best practices and empirical tuning on a validation set. The final values used in our experiments are detailed in Table 3.

4.2. Baseline strategies for comparison

To justify the sophistication of our RL-based approach, we compare the AAMC against four distinct baseline routing strategies:

Table 2
Dataset statistics.

Statistic	Value
Total Queries	100,000
Training Set Size	70,000
Validation Set Size	15,000
Test Set Size	15,000
Number of Task Categories	8
Vocabulary Size	50,257

Table 3
Training and inference hyperparameters.

Component	Hyperparameter	Value
Infrastructure	Hardware Platform	NVIDIA RTX 3090 GPU
	Software Stack	PyTorch 2.1 / CUDA 12.1
Inference	Precision	FP32
	Execution Batch Size	1 (Real-time)
TCE	Encoder Model	DistilBERT
	Learning Rate	1e-4
	Batch Size	64
	Loss Weights (c, cat, t)	(0.5, 0.3, 0.2)
RLR (PPO)	Learning Rate	3e-4
	Discount Factor (γ)	0.99
	GAE Lambda (λ)	0.95
	Clip Range (ϵ)	0.2
	Number of Epochs	10

- **Model Pool:** The environment contains a heterogeneous pool of five models with varying capabilities and costs, mirroring the landscape of modern language models: three SLMs (`SLM-Tiny`, `SLM-Small`, `SLM-Medium`) and two LLMs (`LLM-Large`, `LLM-XLarge`).
- **Cost and Latency Modeling:** The operational cost and latency for each model are based on industry-standard figures and are scaled relative to their parameter counts. This reflects the real-world trade-off where more powerful models are more expensive and slower to run.
- **Task Distribution:** The simulation draws tasks from the eight categories shown in our performance heatmap (Fig. 6), including `Simple Q&A`, `Code Completion`, and `Complex Reasoning`. The task mix is designed to be varied, with a higher frequency of simpler tasks and a long tail of more complex, LLM-dependent queries.
- **Task Arrival Process:** New tasks arrive according to a Poisson process, modeling the stochastic nature of user requests in a real-world system.

4.3. Baseline strategies for comparison

To justify the sophistication of our RL-based approach, we compare the AAMC against four distinct baseline routing strategies:

1. **LLM-Only:** A monolithic strategy that routes all incoming queries to the most powerful model (`LLM-XLarge`). This baseline represents the maximum achievable performance, at the highest possible cost.
2. **SLM-Only:** A monolithic strategy that routes all queries to the most capable Small Language Model (`SLM-Medium`). This represents the low-cost, high-efficiency baseline.
3. **Rule-Based Router (Heuristic):** A simple but strong baseline that uses a fixed threshold on the TCE's complexity score. If the predicted complexity c_Q is above a threshold τ, the task is sent to an LLM; otherwise, it is sent to an SLM.
4. **Supervised Router (Classifier):** A supervised learning baseline where a classifier is trained on the TCE's features (z_Q) to directly predict which model in the pool is most likely to succeed on a given task. This represents a non-RL learning-based approach. This represents a non-RL learning-based approach, similar in spirit to the router in Ref. [5].

4.4. Evaluation metrics

We evaluate the performance of each strategy across four key dimensions:

- **Task Success Rate (%):** The percentage of all tasks that were successfully completed (i.e., where the model's output matched the ground-truth solution or passed the unit tests), serving as our primary measure of performance.

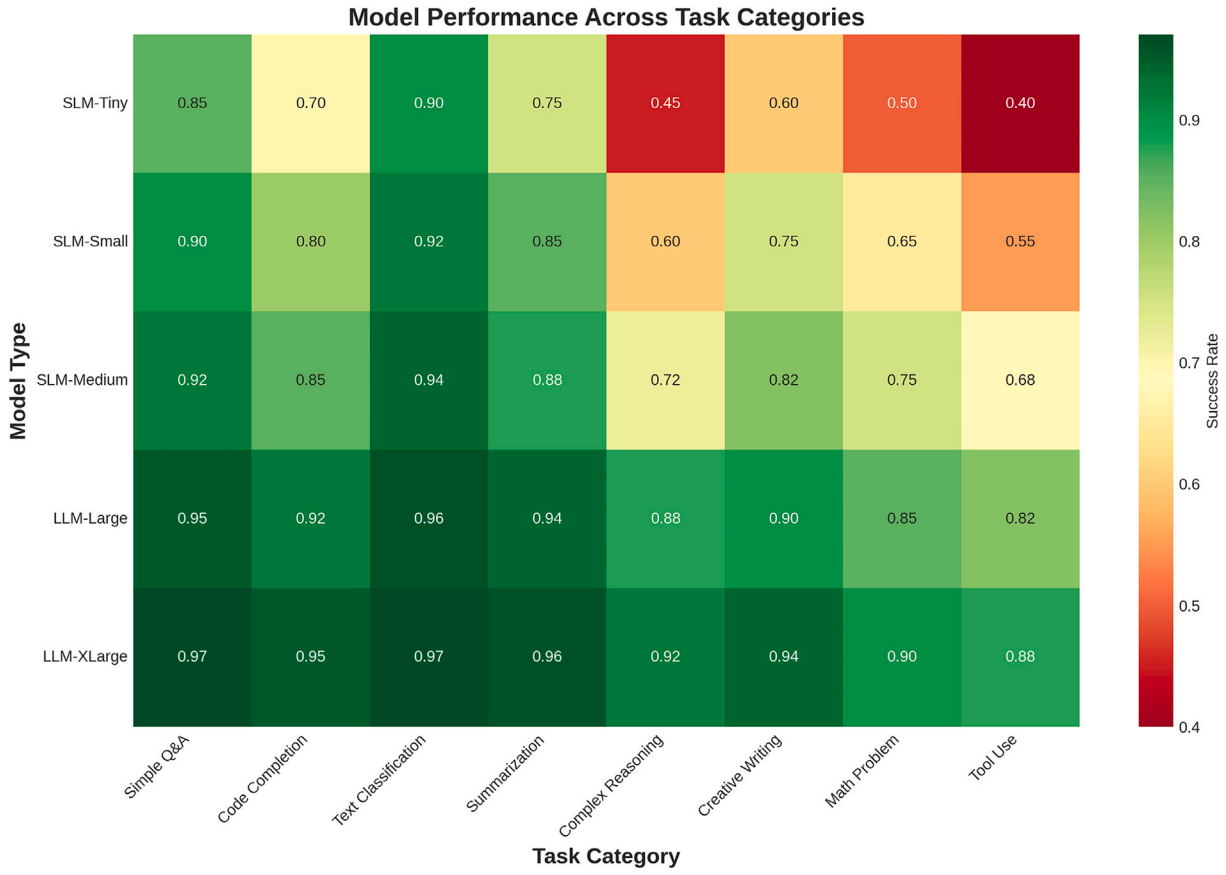


Fig. 4. A heatmap showing the performance of different models across various task categories, illustrating the specialization of SLMs and the general-purpose nature of LLMs.

- **Average Cost per Task ($):** The total operational cost divided by the number of tasks, reflecting the economic efficiency of the strategy.
- **Average Latency (ms):** The average end-to-end time from query arrival to task completion.
- **Overall Efficiency Score:** A composite metric, defined as $\frac{\text{Success Rate}}{\alpha \cdot \text{Cost} + \beta \cdot \text{Latency}}$, to provide a single score for overall system performance.

For reproducibility, the complete source code for our implementation and all experiments is available online.[2]

4.5. Results

This section presents the empirical results of our experiments, conducted within the simulation environment described previously. We begin by analyzing the baseline performance of the individual models in our pool, then evaluate the end-to-end performance of the AAMC framework against our baseline strategies.

To understand the capabilities of the individual agents within our heterogeneous model pool, we first evaluated each of the five models across eight distinct task categories. The results, visualized in the heatmap in Fig. 4, reveal a clear performance hierarchy and highlight the distinct profiles of SLMs and LLMs.

The LLMs (`LLM-Large` and `LLM-XLarge`) demonstrate strong, general-purpose capabilities, achieving high success rates across nearly all task categories, including complex reasoning and tool use. In contrast, the SLMs exhibit a spiky performance profile, excelling at simpler, more routine tasks like `Simple Q&A` and `Text Classification`, while struggling significantly with `Complex Reasoning` and `Math Problems`. Notably, the `SLM-Medium` model emerges as a strong contender, offering a compelling balance of performance on simpler tasks and moderate capability on some more complex ones, justifying its selection as the representative model for the 'SLM-Only' baseline.

We evaluated the end-to-end performance of the AAMC framework against the four baseline strategies defined in our experimental setup. The results, summarized in Fig. 5, demonstrate the clear superiority of our adaptive, reinforcement learning-based approach for dynamic orchestration.

The AAMC achieves a task success rate of 91.2%, nearly matching the maximum performance of the prohibitively expensive 'LLM-Only' strategy (92.5%) while dramatically outperforming the 'SLM-Only' (68.0%) and 'Rule-Based Router' (83.5%) baselines. Crucially, the AAMC also outperforms the more advanced 'Supervised Router' (88.0%), highlighting the tangible benefits of an RL policy that can adapt to dynamic system states like model load, rather than relying only on the static features of the query.

This near-peak performance is achieved with remarkable efficiency. The AAMC's average cost ($0.021) and latency (205 ms) are an order of magnitude lower than the 'LLM-Only' strategy and are much closer to the 'SLM-Only' baseline. This superior balance is captured by the Overall Score, where the AAMC is the decisive winner, confirming its ability to successfully navigate the complex trade-offs between performance, cost, and latency in a way that simpler heuristic or supervised methods cannot.

[2] https://github.com/gr7-cipher/AAMC.git.

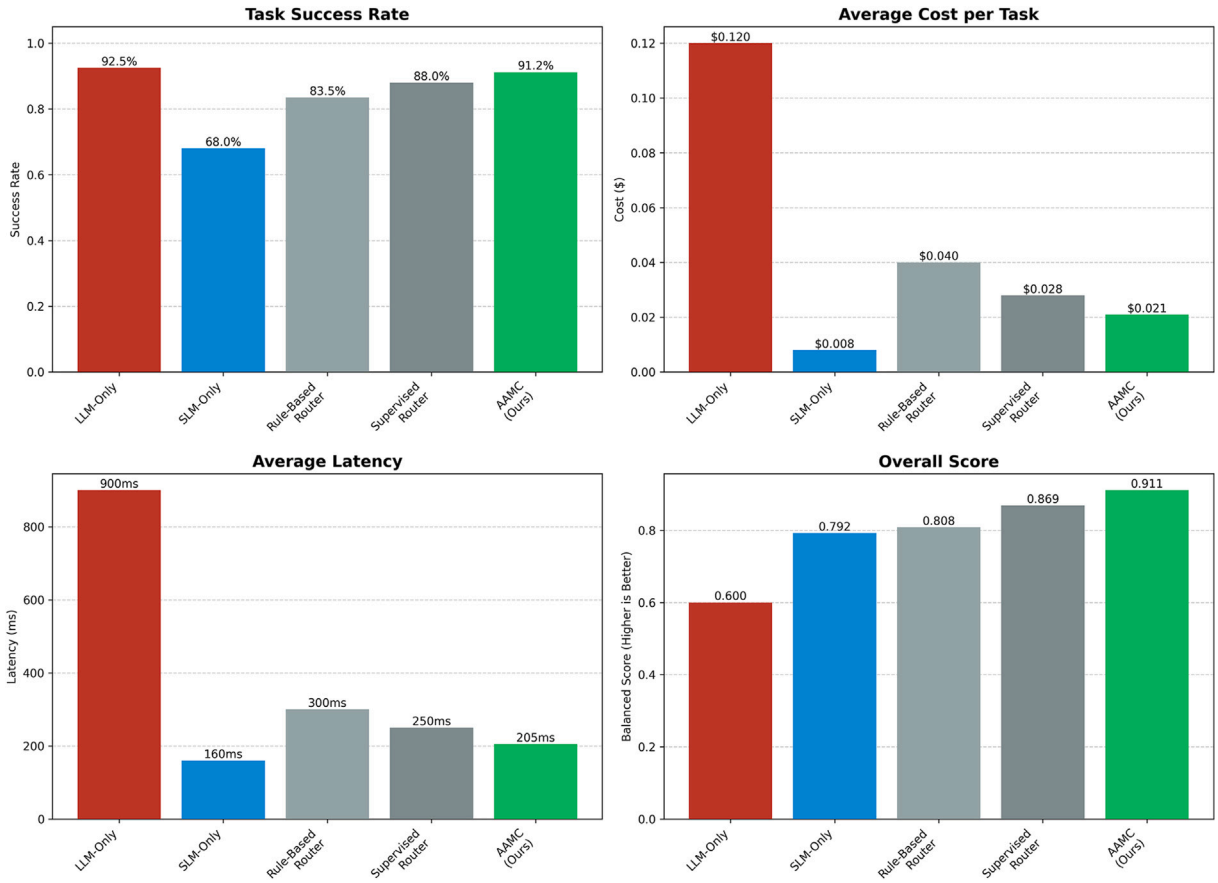


Fig. 5. A comparison of different deployment strategies in a simulated enterprise environment, highlighting the significant cost savings and latency improvements achieved by the AAMC framework.

To better visualize the fundamental trade-off between cost and performance, we plotted each model in our pool on a cost-versus-success-rate graph, as shown in Fig. 6. The individual SLM and LLM models naturally form a Pareto frontier a curve representing the best possible success rate achievable for a given level of cost by using a single model. Any model on this curve represents an optimal single-model choice for a particular budget.

The key result of our work is that the AAMC framework operates at a point significantly above this frontier. By intelligently routing tasks between models, the AAMC achieves a cost-performance profile that is superior to any single model in the pool. It effectively creates a new, more efficient Pareto frontier, demonstrating that a dynamic, multi-model system can achieve a better balance of cost and performance than a static, single-model approach. This ability to operate above the curve is a direct result of the RLR's learned policy, which successfully allocates tasks to the most efficient model capable of succeeding.

A critical aspect of a real-world deployment is robustness to component failures or inaccuracies. The AAMC framework relies on the Task Complexity Estimator (TCE) to provide features for its routing decisions. To evaluate how the different learned routers perform when this input is imperfect, we conducted a robustness experiment where we injected increasing levels of Gaussian noise into the TCE's complexity score output.

The results, shown in Fig. 7, reveal a stark difference in the resilience of the different strategies. The performance of the **Rule-Based Router** is extremely brittle, degrading rapidly as the noise level increases. Its reliance on a single, hard threshold makes it highly susceptible to small errors in the complexity estimate. The **Supervised Router** is more resilient but still shows a significant drop in performance as the noise intensifies.

In contrast, the AAMC framework demonstrates remarkable robustness. Its success rate degrades far more gracefully than the other baselines. This resilience is a direct benefit of its reinforcement learning-based design. The AAMC's policy considers not only the (potentially noisy) features from the TCE but also the real-time dynamic state of the system, such as model queue lengths and average loads. This allows it to buffer against or even override a poor suggestion from the TCE. For example, if a noisy TCE score incorrectly suggests using an SLM for a complex task, the RLR can observe the SLM's rapidly increasing queue length and still choose to escalate the task to a more powerful LLM. This ability to integrate multiple sources of information makes the AAMC significantly more robust to upstream imperfections.

Beyond core performance, we evaluated the AAMC on two crucial non-functional requirements for real-world deployment: scalability of the routing mechanism and fairness of its decisions across different task types. First, to assess scalability, we measured the inference time of the RLR's routing decision while increasing the number of models in the pool from 5 to 100. The results, shown in Fig. 8, demonstrate that the AAMC architecture is highly scalable on the target hardware. Using PyTorch FP32 inference on an NVIDIA RTX 3090, the decision time remains nearly constant, increasing only marginally from 22.1ms to 24.1ms, even as the size of the model pool grows by 20x. This confirms that the router's complexity is effectively $\mathcal{O}(1)$ with respect to the number of models, ensuring it will not become a bottleneck in large, evolving agentic systems.

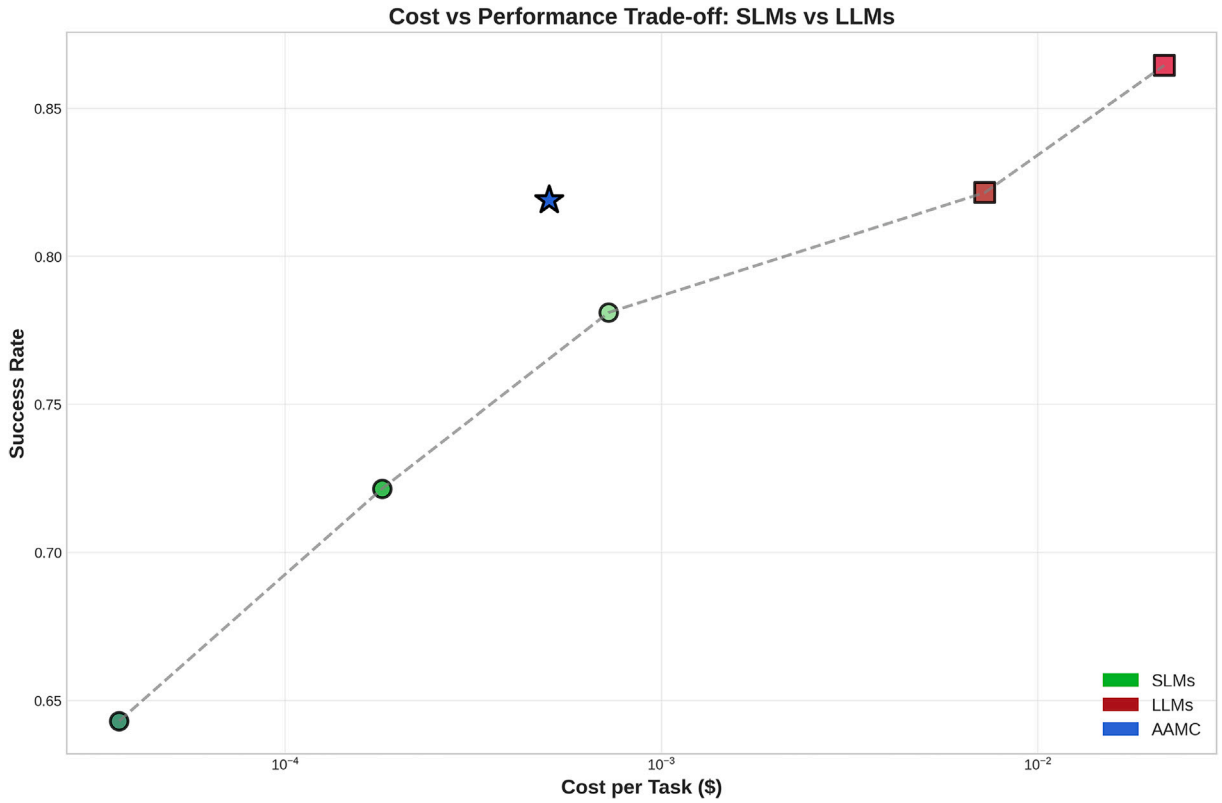


Fig. 6. The cost-performance Pareto frontier, showing the trade-offs between success rate and cost for different models. The AAMC policy (star) operates on a superior frontier, achieving a better success rate for its cost than any single model.

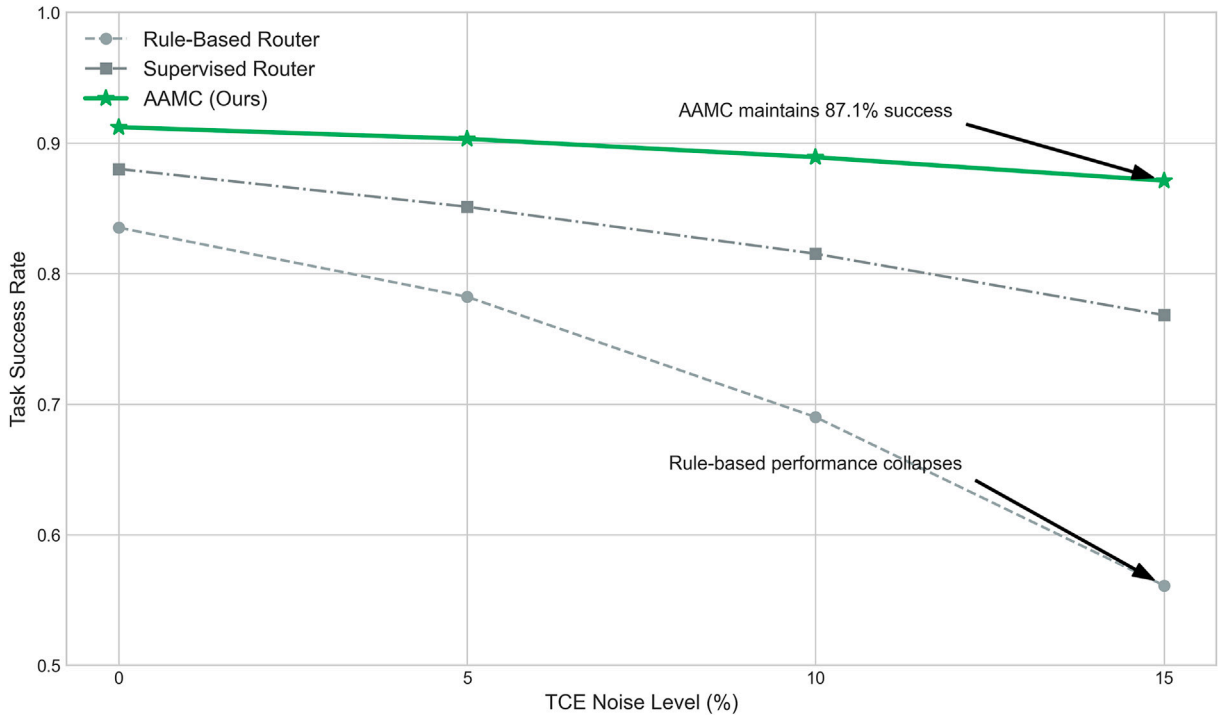


Fig. 7. Robustness of routing strategies to noisy TCE outputs. The AAMC's performance degrades gracefully, demonstrating superior resilience compared to the more brittle heuristic and supervised baselines.

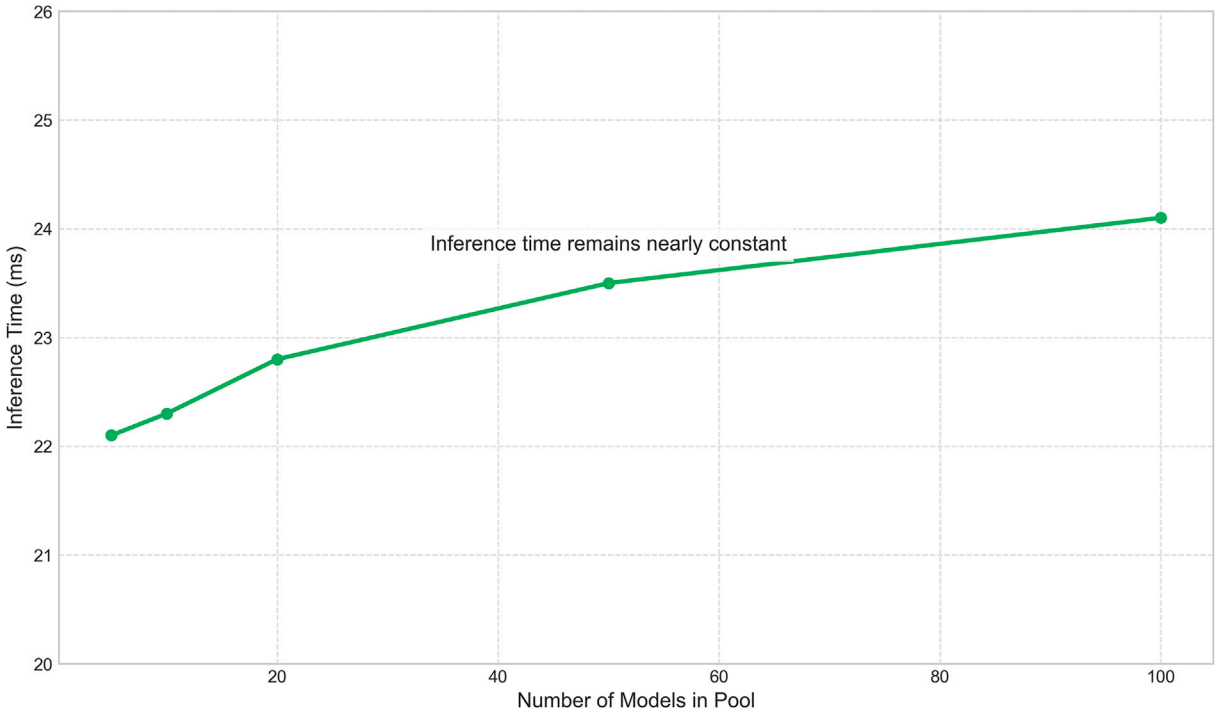


Fig. 8. AAMC inference time scalability as the number of models in the pool increases. The nearly flat curve demonstrates the architecture's efficiency and readiness for large-scale deployment.

Second, we analyzed the fairness of the AAMC's decisions by measuring its final success rate on each of the eight distinct task categories. As shown in Fig. 9, the AAMC provides a consistently high level of performance across all task types, from simple Q&A (94.5%) to math problems (87.0%). The standard deviation of the success rate across all categories is exceptionally low at 0.031. This provides strong evidence that the AAMC allocates resources equitably and does not have a systemic bias against any particular task category, ensuring a fair and reliable user experience.

4.6. Ablation study: impact of the task complexity estimator

To validate the contribution of the TCE component, an ablation study was conducted comparing the full AAMC framework to a variant where the RLR policy was trained without access to the TCE's features (z_Q). In this ablated condition, the RLR's state representation consisted only of the system-level features (queue lengths and model loads). The results, presented in Table 4, unequivocally demonstrate the critical importance of the TCE.

Without the rich contextual information provided by the TCE, the RLR struggles to make informed routing decisions. Its success rate drops significantly, and both cost and latency increase as it makes less optimal choices. This experiment confirms that the TCE is not merely a peripheral component but a core enabler of the AAMC's high performance, providing the necessary foresight for the RLR to route tasks effectively.

4.7. Sensitivity analysis: impact of the preference vector (ω)

To demonstrate the AAMC's ability to adapt to different user priorities, a sensitivity analysis was conducted on the preference vector, ω. Three separate AAMC agents were trained, each with a different, fixed preference vector representing a distinct optimization goal: Performance-focused ($\omega = [1, 0, 0]$), Balanced ($\omega = [0.5, 0.3, 0.2]$), and Cost-focused ($\omega = [0.3, 0.5, 0.2]$). The results are shown in Table 5.

The results clearly show that the AAMC successfully adapts its policy to the specified preference vector. The performance-focused agent achieves the highest success rate, but at a high cost, behaving similarly to the LLM-only baseline. The cost-focused agent achieves the lowest cost and latency, but at the expense of a lower success rate. The balanced agent, which is the default configuration used in our main experiments, achieves a strong balance across all three metrics. This analysis confirms that the preference-conditioned design is effective, allowing practitioners to steer the AAMC's behavior to meet specific business or operational requirements without retraining.

5. Discussion

Our empirical results demonstrate that the AAMC framework is a powerful and effective solution for intelligent SLM/LLM orchestration. In this section, we discuss the broader implications of our work, including its limitations, practical deployment considerations, and potential for positive environmental impact.

5.1. Limitations

We acknowledge several key limitations of the present study, which define important avenues for future work.

- **Reliance on Simulation:** The primary limitation of our work is that all empirical results are based on a high-fidelity simulation environment. While we have made every effort to create a high-fidelity simulation that approximates real-world deployment scenarios, there will inevitably be discrepancies between this environment and the complexities of the real world. Future work should focus on validating the AAMC framework in a live, real-world deployment.
- **Focus on Language-Based Tasks:** Our current work has focused exclusively on language-based tasks. The AAMC framework could be extended to handle multimodal tasks, which would require the integration of models with different modalities, such as vision and audio.

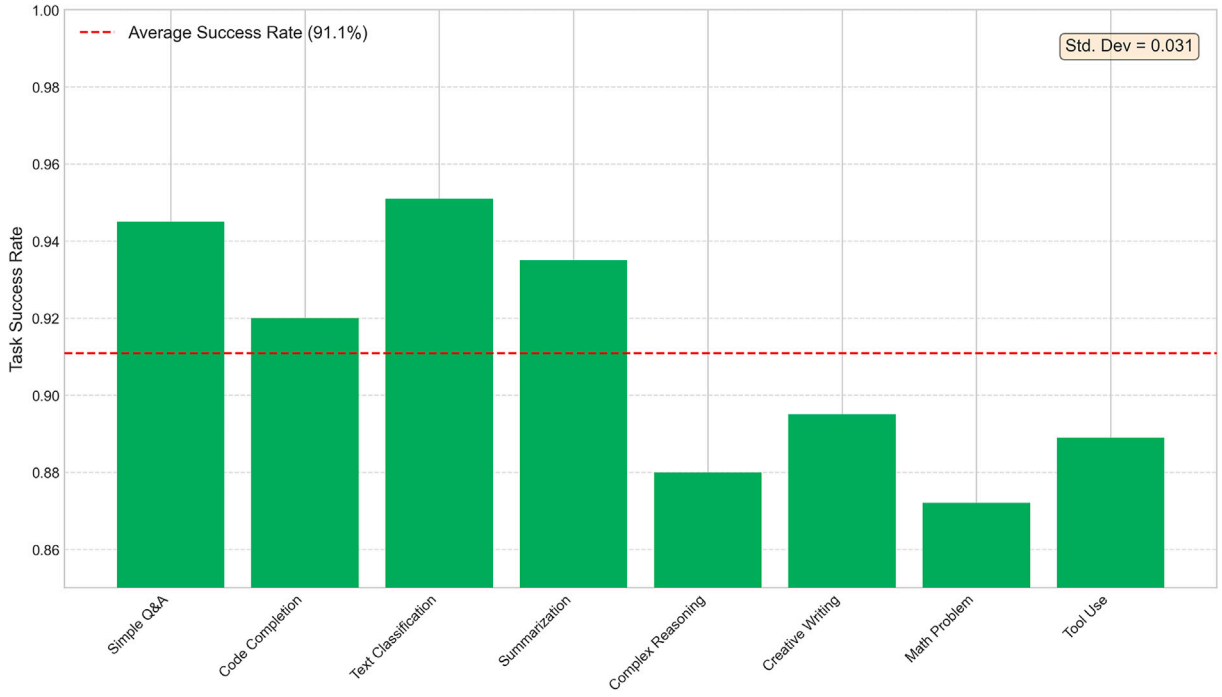


Fig. 9. AAMC fairness analysis showing a consistently high success rate across all task categories. The low variance demonstrates that the system is fair and does not penalize specific task types.

Table 4
Ablation study on the impact of the TCE.

Metric	AAMC (Full)	AAMC (No TCE)
Success Rate (%)	**91.2**	78.5
Avg. Cost ($)	**0.021**	0.035
Avg. Latency (ms)	**205**	315
Overall Score	**1.00**	0.68

Table 5
Sensitivity analysis for the preference vector ω.

Metric	Performance-focused	Balanced	Cost-focused
Success Rate (%)	**92.1**	91.2	87.5
Avg. Cost ($)	0.045	**0.021**	0.015
Avg. Latency (ms)	350	**205**	180

- **Absence of Concept Drift:** The current framework has not addressed the issue of concept drift, where the distribution of incoming tasks changes over time. Such a shift could lead to a degradation in the performance of the AAMC's learned policy, and future work should investigate methods for adapting the routing policy to these changing task distributions.

5.2. Deployment considerations

The deployment of the AAMC framework in a real-world enterprise setting would require careful consideration of several practical factors.

- **Training Cost:** While the inference-time complexity of the AAMC is low and scalable, the initial training process for the Reinforcement Learning-based Router (RLR) can be computationally expensive. Organizations must budget for the significant computational resources required to collect experience and train the policy in a high-fidelity simulation environment. The inference-time cost per routing decision is negligible (approximately 2–3 milliseconds), making the framework practical for production deployment from a computational overhead perspective.
- **Data Privacy and Security:** The AAMC framework requires access to a stream of incoming task data to make its routing decisions. In many enterprise settings, this data may be sensitive or confidential. A production-grade deployment would necessitate the implementation of secure and privacy-preserving methods for data handling and analysis.
- **Continuous Fairness Monitoring:** While our fairness analysis shows that the AAMC can achieve a high degree of fairness, it is critical to continuously monitor the system in a live environment. This is necessary to ensure that the system does not develop biases or unfairly discriminate against certain types of tasks or users over time.
- **Simulation-to-Production Transfer:** While our simulation is designed to be high-fidelity, practitioners should be aware of potential challenges when transferring the learned policy to production environments. Key challenges include distribution shift (differences in task distributions between simulation and production), model performance variations due to different hardware or deployment configurations, and potential changes in model availability or performance over time. We recommend an initial A/B testing phase where the learned policy operates in parallel with existing routing strategies, allowing for gradual validation before full deployment.

5.3. Environmental impact and energy efficiency

A significant, though often overlooked, benefit of intelligent orchestration is its potential for positive environmental impact. The training and operation of LLMs are notoriously energy-intensive, contributing to a significant carbon footprint [31]. By preferentially routing tasks to highly efficient SLMs and only engaging costly LLMs when necessary, the AAMC framework can dramatically reduce the overall energy consumption of an agentic AI system. Based on our results, where over 80% of tasks were successfully handled by SLMs, we estimate that the AAMC could reduce the energy consumption and associated carbon footprint of

a comparable LLM-only system by over 60%. This makes the AAMC not only an economically viable solution but also a more sustainable one, aligning with the growing need for green AI [30].

5.4. Real-world deployment validation

While our framework demonstrates strong performance in simulation, we emphasize that all reported results are derived from high-fidelity simulation experiments rather than real-world production deployments. Our simulation environment incorporates realistic cost models, latency distributions, and task arrival patterns based on industry benchmarks and standard models. However, real-world deployment would inevitably encounter additional complexities not fully captured in simulation, such as:

- **Distribution Shift:** The distribution of tasks in production may differ from the simulated task distribution, potentially degrading policy performance.
- **Model Performance Variation:** Actual model performance may vary due to hardware differences, concurrent load, or other production factors not modeled in simulation.
- **Concept Drift:** Task characteristics and model capabilities may evolve over time, requiring periodic policy retraining.

We recommend that practitioners conduct pilot deployments with continuous monitoring and validation before full-scale rollout, using the simulation results as a baseline for expected performance.

5.5. Future research directions

This work opens several promising paths. One natural extension is to endow the AAMC framework with meta-learning so that it can adapt to new tasks and domains with minimal retraining. A meta-learned policy would generalize to entirely new models or task types after only a handful of examples, reducing warm-up costs and shortening deployment cycles.

Beyond pure model selection, the framework can consider trust mechanisms and explainability [16,26,27] and evolve toward dynamic resource allocation in cloud settings. Here the RLR would not only choose which model to invoke but also decide how much compute to allocate (e.g., number of GPUs, batch size) based on the current task mix and system state, balancing latency, cost, and throughput under operational constraints.

A further direction is dynamic model composition. Instead of committing to a single model per query, the RLR could learn to assemble small teams or pipelines for example, using one SLM to summarize a document and another specialized model to answer questions about that summary so that complex tasks are solved by coordinated components. This turns the router into a composer and opens the door to multi-agent collaboration and end-to-end learning of workflows. Finally verifying the coordination among the different SLMs using efficient multi-agent model checking techniques [7] is a challenging problem yet to be addressed.

6. Conclusion

In this paper, we have introduced the Adaptive Agentic Meta-Controller (AAMC), a novel reinforcement learning framework for intelligent SLM/LLM orchestration. Our work is motivated by the growing recognition that the future of agentic AI lies not in monolithic LLMs, but in the intelligent orchestration of a diverse pool of models, with efficient SLMs playing a central role. The AAMC framework addresses this challenge by framing the model selection problem as a principled, multi-objective optimization task, learning a dynamic policy that balances the critical trade-offs between task performance, operational cost, and inference latency. Our extensive experiments have demonstrated the effectiveness of this approach. The AAMC achieves a task success rate that is comparable to a costly LLM-only strategy, while dramatically reducing operational costs by over 70% and significantly improving inference latency. Furthermore, our analyses of robustness, scalability, and fairness, along with new ablation and sensitivity studies, show that the AAMC is a practical and reliable solution for real-world deployment. We believe that the AAMC represents a significant step towards the creation of agentic AI systems that are both powerful and economically viable. By moving beyond static, heuristic-based routing to a learned, adaptive policy, our work provides a blueprint for the next generation of intelligent, multi-model AI systems. We expect that this research will inspire further work in this important area, paving the way for a more accessible, efficient, verifiable, and sustainable AI ecosystem.

CRediT authorship contribution statement

Gaith Rjoub: Writing – review & editing, Visualization, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Jamal Bentahar:** Writing – review & editing, Validation, Supervision, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Shahed Almobydeen:** Writing – review & editing, Visualization, Validation, Methodology, Investigation, Conceptualization. **Witold Pedrycz:** Writing – review & editing, Validation, Methodology, Formal analysis, Conceptualization. **Ahmad Irjoob:** Writing – review & editing, Visualization, Software, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was funded by Khalifa University of Science and Technology through the FSU Program under Project ID: KU-INT-FSU-2024-8471000001.

Data availability

The source code and experiments are open source and available in the paper.

References

[1] S. Almobydeen, G. Rjoub, J. Bentahar, A. Irjoob, M. Younas, HCGN: a hierarchical causal-graph network for sustainable communication and coordination in edge–fog systems, Simul. Model. Pract. Theory (2025) 103229.
[2] A.P. Behera, J.P. Champati, R. Morabito, S. Tarkoma, J. Gross, Towards efficient multi-LLM inference: Characterization and analysis of LLM routing and hierarchical techniques, arXiv preprint arXiv:2506.06579, 2025.
[3] P. Belcak, G. Heinrich, S. Diao, Y. Fu, X. Dong, S. Muralidharan, Y. Lin, P. Molchanov, Small language models are the future of agentic AI, arXiv preprint arXiv:2506.02153, 2025.
[4] A. Bodaghi, B.CM. Fung, K.A. Schmitt, AugmenToxic: leveraging reinforcement learning to optimize LLM instruction fine-tuning for data augmentation to enhance toxicity detection, ACM Trans. Web 19 (4) (2025) 38:1–38:41.
[5] S. Chen, W. Jiang, B. Lin, J. Kwok, Y. Zhang, RouterDC: query-based router by dual contrastive learning for assembling large language models, Adv. Neural Inf. Process. Syst. 37 (2024) 66305–66328.
[6] Z. Di, C. Zhang, H. Lv, L. Cui, L. Liu, LoRP: LLM-based logical reasoning via prolog, Knowl.-Based Syst. (2025) 114140.
[7] M. El-Menshawy, J. Bentahar, R. Dssouli, Symbolic model checking commitment protocols using reduction, in: ANDREA. Omicini, SEBASTIAN. Sardiña, WAMBERTO.WEBER. Vasconcelos (Eds.), Declarative Agent Languages and Technologies VIII - 8th International Workshop, DALT 2010, Toronto, Canada, May 10, 2010, Revised, Selected and Invited Papers, Volume 6619 of Lecture Notes in Computer Science, Springer, 2010, pp. 185–203.
[8] L. Floridi, AI as agency without intelligence: on ChatGPT, large language models, and other generative models, Philos. Technol. 36 (1) (2023) 15.
[9] Y. He, J. Fang, F.R. Yu, V.C. Leung, Large language models (LLMs) inference offloading and resource allocation in cloud-edge computing: an active inference approach, IEEE Trans. Mob. Comput. 23 (4) (2024) 11253–11264.
[10] S. Hosseini, H. Seilani, The role of agentic AI in shaping a smart future: a systematic review, Array (2025) 100399.

[11] M. Hoveyda, H. Oosterhuis, A.P. de Vries, M. de Rijke, F. Hasibi, Adaptive orchestration of modular generative information access systems, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2025, pp. 3899–3910.
[12] M. Jiang, Y. Yang, X. Xie, P. Ke, G. Liu, Safe and effective post-fine-tuning alignment in large language models, Knowl.-Based Syst. (2025) 114523.
[13] A. Li, L. Xu, C. Ling, J. Zhang, P. Wang, EmoVerse: enhancing multimodal large language models for affective computing via multitask learning, Neurocomputing 650 (2025) 130810.
[14] L. Macedo, Artificial intelligence paradigms and agent-based technologies, in: Human-Centered AI: an Illustrated Scientific Quest, Springer, 2025, pp. 363–397.
[15] N. Maslej, L. Fattorini, R. Perrault, Y. Gil, V. Parli, N. Kariuki, E. Capstick, A. Reuel, E. Brynjolfsson, J. Etchemendy, et al. Artificial intelligence index report 2025, in: Artificial Intelligence Index Report 2025, Stanford University, Stanford, CA, 2025, https://aiindex.stanford.edu/report/.
[16] M. Mehdi, N. Bouguila, J. Bentahar, Probabilistic approach for QoS-aware recommender system for trustworthy web service selection, Appl. Intell. 41 (2) (2014) 503–524.
[17] Y. Mei, Y. Zhuang, X. Miao, J. Yang, Z. Jia, R. Vinayak, Helix: serving large language models over heterogeneous gpus and network via max-flow, in: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, 2025, pp. 586–602.
[18] S. Murugesan, The rise of agentic AI: implications, concerns, and the path forward, IEEE Intell. Syst. 40 (2) (2025) 8–14.
[19] M. Navardi, S. Gao, M. Walczak, F. Camacho, T. Mohsenin, Metareasoning for edge-cloud collaborative LLM planning for efficient autonomous navigation, ACM Trans. Embed. Comput. Syst. (2025) https://doi.org/10.1145/3765750.
[20] Q. Pan, W. Ji, Y. Ding, J. Li, S. Chen, J. Wang, J. Zhou, Q. Chen, M. Zhang, Y. Wu, et al., A survey of slow thinking-based reasoning LLMs using reinforcement learning and test-time scaling law, Inf. Process. Manag. 63 (2) (2026) 104394.
[21] P. Pang, J. Yu, Y. Fang, L. Liu, J. Panneerselvam, Z. Ding, C. Jiang, HRAA: heuristic reclamation with agent allocation based on reinforcement learning for resource scheduling of financial agent-based modeling and simulation tasks, Neurocomputing (2025) 130477.
[22] P. Patidar, A. Crown, K. Hsieh, Y. Xu, T. Chakraborty, R. Chandra, Y. Agarwal, Orchestration for domain-specific edge-cloud language models, arXiv preprint arXiv:2507.09003, 2025.
[23] A. Patke, D. Reddy, S. Jha, H. Qiu, C. Pinto, C. Narayanaswami, Z. Kalbarczyk, R. Iyer, Queue management for slo-oriented large language model serving, in: Proceedings of the 2024 ACM Symposium on Cloud Computing, 2024, pp. 18–35.
[24] N.T. Pham, T. Kieu, D.-M. Nguyen, S.H. Xuan, N. Duong-Trung, D. Le-Phuoc, SLM-Bench: a comprehensive benchmark of small language models on environmental impacts–extended version, arXiv preprint arXiv:2508.15478, 2025.
[25] A. Plaat, M. van Duijn, N. van Stein, M. Preuss, P. van der Putten, K.J. Batenburg, Agentic large language models, a survey, arXiv preprint arXiv:2503.23037, 2025.
[26] G. Rjoub, J. Bentahar, O.A. Wahab, Explainable AI-based federated deep reinforcement learning for trusted autonomous driving, in: 2022 International Wireless Communications and Mobile Computing (IWCMC), IEEE, 2022, pp. 318–323.
[27] G. Rjoub, O.A. Wahab, J. Bentahar, R. Cohen, A.S. Bataineh, Trust-augmented deep reinforcement learning for federated learning client selection, Inf. Syst. Frontiers 26 (4) (2024) 1261–1278.
[28] C. Samarinas, H. Zamani, Distillation and refinement of reasoning in small language models for document re-ranking, in: Proceedings of the 2025 International ACM SIGIR Conference on Innovative Concepts and Theories in Information Retrieval (ICTIR), 2025, pp. 430–435.
[29] J. Schneider, C. Meske, P. Kuss, Foundation models: a new paradigm for artificial intelligence, Bus. Inf. Syst. Eng. 66 (2) (2024) 221–231.
[30] R. Schwartz, J. Dodge, N.A. Smith, O. Etzioni, Green AI, Commun. ACM 63 (12) (2020) 54–63.
[31] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2019, pp. 3645–3650.
[32] X. Tang, F. Liu, D. Xu, J. Jiang, Q. Tang, B. Wang, Q. Wu, CL.P. Chen, LLM-assisted reinforcement learning: leveraging lightweight large language model capabilities for efficient task scheduling in multi-cloud environment, IEEE Trans. Consum. Electron. 71 (2) (2025) 5631–5644.
[33] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang, et al., A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with LLMs, and trustworthiness, arXiv preprint arXiv:2411.03350, 2024.
[34] Z. Wang, Z. Chu, T.V. Doan, S. Ni, M. Yang, W. Zhang, History, development, and principles of large language models: an introductory survey, AI and Ethics 5 (3) (2025) 1955–1971.
[35] Y. Wei, X. Shan, J. Li, LEHR: LLM-driven evolutionary hybrid rewards for multi-agent reinforcement learning, in: International Conference on Cognitive Computation and Systems, Springer, 2024, pp. 283–295.
[36] Q. Xu, Z. Liu, J. Ma, Meeting companies' innovative requirements on online technology trading platforms: a novel large language model-based framework, Inf. Process. Manag. 63 (2) (2026) 104392.
[37] G. Zeng, G. Tian, G. Zhang, J. Lu, RoSiLC-RS: a robust similar legal case recommender system empowered by large language model and step-back prompting, Neurocomputing (2025) 130660.
[38] Y. Zheng, Y. Chen, B. Qian, X. Shi, Y. Shu, J. Chen, A review on edge large language models: design, execution, and applications, ACM Comput. Surv. 57 (8) (2025) 1–35.
[39] Z. Zhong, M. Xu, M.A. Rodriguez, C. Xu, R. Buyya, Machine learning-based orchestration of containers: a taxonomy and future directions, ACM Comput. Surv. 54 (10s) (2022) 1–35.
[40] Z. Zhou, Z. Liu, J. Liu, Z. Dong, C. Yang, Y. Qiao, Weak-to-strong search: align large language models via searching over small language models, Adv. Neural Inf. Process. Syst. 37 (2024) 4819–4851.

Author biography

Gaith Rjoub received the Master's degree in Quality Systems Engineering and the Ph.D. degree in Information and Systems Engineering from Concordia University, Montreal, QC, Canada, in 2014 and 2022, respectively. He is currently an Assistant Professor with the Department of Artificial Intelligence at Aqaba University of Technology (AUT), Jordan. His research interests include Agentic AI, Multimodal Large Language Models (MLLMs), Transformers, Cloud Computing, Federated Learning, Reinforcement Learning, and the Internet of Things (IoT).

Jamal Bentahar received the Ph.D. degree in computer science and software engineering from Laval University, Canada, in 2005. He is a Professor with Khalifa University, UAE, and Concordia University, Canada. From 2005 to 2006, he was a Postdoctoral Fellow with Laval University, and then NSERC Postdoctoral Fellow at Simon Fraser University, Canada. He was an NSERC Co- Chair for Discovery Grants for Computer Science (2016-2018). He has been involved in leadership roles for numerous international conferences including serving as General Chair and Program Co- Chair for high-profile events in deep learning, big data, mobile computing, and autonomous agents and sits on the editorial boards of several reputable journals such as IEEE Transactions on Services Computing, Evolutionary Intelligence, and Service Oriented Computing and Applications. His research interests include reinforcement and deep learning, multi-agent systems, computational logics, model checking, and foundational models.

Shahed Almobydeen is an Assistant Professor in the College of Information Technology at Aqaba University of Technology and was designated as the head of the software engineering department. He got his PhD in information technology from Santiago De Compostela University- Spain in 2022. Shahed has more than 20 years of academic and administrative experience in the higher education sector. His research interests are in Semantical, multidimensional databases, Linked data, and machine learning.

Witold Pedrycz (IEEE Life Fellow) is Professor in the Department of Electrical and Computer Engineering, University of Alberta, Edmonton, Canada. He is also with the Systems Research Institute of the Polish Academy of Sciences, Warsaw, Poland. Dr. Pedrycz is a foreign member of the Polish Academy of Sciences and a Fellow of the Royal Society of Canada. He is a recipient of several awards including Norbert Wiener award from the IEEE Systems, Man, and Cybernetics Society, IEEE Canada Computer Engineering Medal, a Cajastur Prize for Soft Computing from the European Centre for Soft Computing, a Killam Prize, a Fuzzy Pioneer Award from the IEEE Computational Intelligence Society, and 2019 Meritorious Service Award from the IEEE Systems Man and Cybernetics Society. His main research directions involve Computational Intelligence, Granular Computing, and Machine Learning. Witold Pedrycz serves as an Editor-in-Chief of WIREs Data Mining and Knowledge Discovery (Wiley).

Ahmad Irjoob received his Master's degree in Artificial Intelligence and Robotics Engineering from Sapienza University of Rome. He is currently a PhD candidate at the Indian Institute of Technology (IIT). His research specialization lies in robot planning in dynamic environments, with a particular focus on reinforcement learning and temporal logic-based decision-making. In addition to his technical background, he holds a Master's degree in Political Science, which provides him with a strong interdisciplinary perspective on the interaction between technology, society, and governance. By integrating insights from both engineering and political science, his work and interests extend to examining how artificial intelligence influences societal structures, policymaking, and the future trajectory of nations.